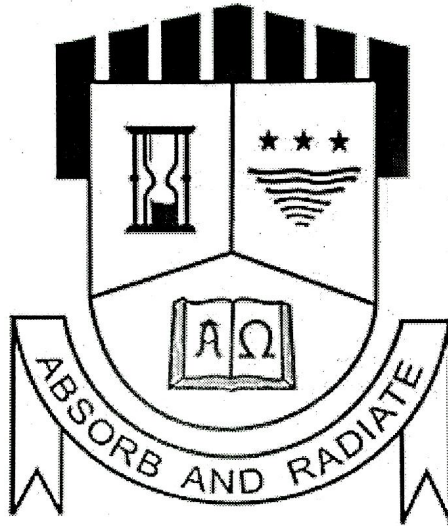


ST. DOMINICS COLLEGE
KANJIRAPPALLY



DEPARTMENT OF PHYSICS

Project Report on
Satellite Visibility and Ground Tracking with Python

Submitted To,
Mahatma Gandhi University

2021-2024

Satellite Visibility and Ground Tracking with Python

Submitted To,

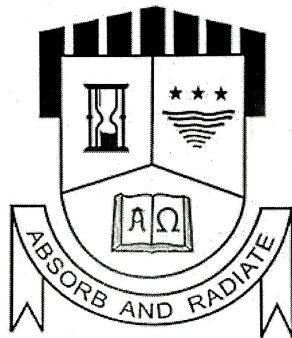
Mahatma Gandhi University

*In partial fulfillment of the requirements for the award
of the Degree of **Bachelor of Science in Physics***

Submitted by,

NICHOL JOSEPH
(Reg.no: 210021034041)

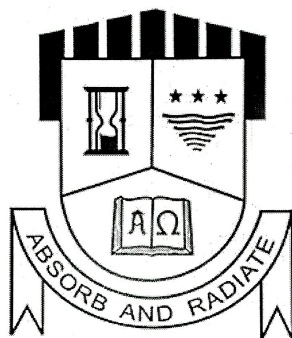
Guided by : Dr. VIMAL G



Department of Physics

St. Dominic's College Kanjirappally

2021-24



Department of Physics

St. Dominic's College Kanjirappally

CERTIFICATE

This is to certify that the project report entitled "**SATELLITE VISIBILITY AND GROUND TRACKING WITH PYTHON**" is an authentic record of the project work carried out by NICHOL JOSEPH, Reg. No: 210021034041 on partial fulfillment of the requirement for the award of the B.Sc. degree in physics of Mahatma Gandhi University, in the Department of Physics, St. Dominic's College Kanjirappally, during the academic year 2021-2024.

SMITHA MATHEW
Head of the Department

Internal Examiner

External Examiner

CERTIFICATE

This is to certify that the project report entitled "**SATELLITE VISIBILITY AND GROUND TRACKING WITH PYTHON**" is an authentic record of the project work carried out by NICHOL JOSEPH, under my guidance during the academic year 2023-2024 for the partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Physics of Mahatma Gandhi University, Kottayam.

Place:

Date:

Dr. Vimal G
Project Guide

DECLARATION

I, **NICHOL JOSEPH**, do hereby declare that project work entitled “**SATELLITE VISIBILITY AND GROUND TRACKING WITH PYTHON**” is a bonafide record of the work carried out by me under the guidance of Dr. VIMAL G , Assistant professor, Department of Physics, St. Dominic's College Kanjirappally, during the academic year 2023-2024 for the partial fulfillment of the requirements for the award of the degree of Bachelor of Science in Physics of Mahatma Gandhi University, Kottayam.

Place:

Date:

Nichol Joseph

Satellite Visibility and Ground Tracking with Python

Abstract

Satellite visibility and ground tracking are fundamental aspects of satellite analysis, pivotal for numerous scientific, commercial, and military pursuits. This project introduces a sophisticated Python program meticulously crafted to explore satellite visibility and ground tracking using authentic data and advanced computational methodologies. By harnessing the power of Two-Line Element (TLE) data and cutting-edge software libraries such as Skyfield and Matplotlib, the program empowers users to delve into the intricate dynamics of satellite motion and predict their trajectories with precision. The primary objective of this project is to democratize access to satellite analysis tools and enable comprehensive investigations into satellite visibility and ground tracking.

Through meticulous analysis of TLE data and the implementation of sophisticated computational techniques, users can gain invaluable insights into satellite behavior, optimize satellite communication and navigation systems, and enhance Earth observation capabilities. Key features of the Python program include the ability to calculate satellite pass times, determine visibility from specific observer locations, and visualize satellite ground tracks over the Earth's surface. By providing users with a user-friendly interface and robust analytical capabilities, the program serves as a catalyst for innovation and discovery in the field of satellite technology. In essence, this project represents a significant step forward in the democratization of space exploration, empowering researchers, educators, and enthusiasts worldwide to unravel the mysteries of satellite motion and unlock new possibilities for scientific inquiry and technological advancement.

Table of Contents

Abstract

1. Introduction

2. Literature Review

3. Methodology

3.1 Data Collection

3.2 Data Processing

3.3 Analysis Techniques

4. Libraries

5. Code Implementation and Explanation

6. Results

7. Discussion

8. Conclusion

9. References

10. Appendices

1. Introduction

In an era dominated by technological advancements, the study of satellite visibility and ground tracking plays a crucial role in various scientific, commercial, and military endeavors. Satellites orbiting the Earth provide vital services such as communication, navigation, remote sensing, and environmental monitoring. Understanding the dynamics of satellite motion and predicting their trajectories are essential tasks for optimizing satellite operations and ensuring efficient utilization of space resources.

The purpose of this project is to develop a Python program capable of analyzing satellite visibility and ground tracking using real-world data and computational techniques. By leveraging Two-Line Element (TLE) data, a standardized format for conveying sets of orbital elements for Earth-orbiting objects, along with advanced software tools like Skyfield and Matplotlib, this program aims to provide users with insights into the behavior and positioning of satellites relative to specific observer locations on Earth.

The analysis of satellite visibility involves determining when and where a satellite is visible from a given observer location. This information is crucial for scheduling satellite communication sessions, optimizing ground station operations, and coordinating satellite-based services. Ground tracking, on the other hand, focuses on predicting the path or ground track of a satellite over the Earth's surface, enabling researchers to visualize and analyze its orbital motion in real-time.

By developing a Python program for satellite visibility and ground tracking, this project seeks to democratize access to satellite data analysis tools and empower researchers, educators, and enthusiasts to explore the fascinating realm of space technology. Through hands-on experimentation and data-driven insights, users can gain a deeper understanding of satellite dynamics, orbital mechanics, and the interconnectedness of space-based systems in our modern world.

2. Literature Review:

Previous research on satellite visibility and ground tracking has focused on theoretical models, observational techniques, and practical applications. Studies have explored the dynamics of satellite motion, including orbital parameters, orbital elements, and ground track prediction algorithms. Computational tools such as Skyfield and STK have been widely used for satellite analysis and visualization. However, there remains a need for user-friendly software solutions that enable researchers to analyze satellite visibility and ground tracks efficiently.

3. Methodology

The Python program developed in this project utilizes publicly available TLE data to analyze satellite visibility and ground tracking. By specifying the observer's latitude and longitude, users can calculate the pass times of multiple satellites and plot their ground tracks over the observer location. The program employs the Skyfield library to parse TLE data and compute satellite positions, and Matplotlib for data visualization.

Celestrak Website:

In addition to the local TLE data, the program also utilized data from the Celestrak website (<https://celestrak.com/>) during development and testing phases. Celestrak provides a comprehensive database of satellite orbital information, including TLE data, which was used to validate and cross-reference the locally stored TLE data. By incorporating data from Celestrak, the program ensures accuracy and reliability in satellite analysis and prediction.

Offline Data Integration:

To ensure the program's functionality in offline environments or where internet connectivity might be limited, the Two-Line Element (TLE) data necessary for satellite analysis was integrated into the program locally. This was achieved by storing the TLE data in a local file named `gp.php`. The `gp.php` file contains the necessary TLE data for the satellites of interest, allowing the program to access and parse this data without requiring an active internet connection.

3.1 Data Collection:

The first step in the analysis involved collecting Two-Line Element (TLE) data for the satellites of interest. TLE data is a standardized format used to convey sets of orbital elements for Earth-orbiting objects, such as satellites. Each set of TLE data consists of two lines of text containing specific parameters that describe the satellite's orbit. The TLE format was developed by NORAD (North American Aerospace Defense Command) and is widely used by satellite tracking systems and software.

The first line of a TLE contains the satellite's name or identifier, along with information about the epoch time (the moment in time when the TLE was calculated or predicted). The second line contains the following orbital elements:

- Line Number: A number indicating the line of the TLE (always "1" for the first line and "2" for the second line).
- Satellite Catalog Number: A unique identifier assigned to the satellite by NORAD.
- Classification: A code indicating the type of satellite orbit (e.g., "U" for unclassified).
- International Designator: A code specifying the year of launch and the launch number of the satellite.
- Epoch Year and Day: The year and fractional day of the epoch time.
- First Time Derivative of the Mean Motion (dN/dt): The rate of change of the satellite's mean motion over time.
- Second Time Derivative of Mean Motion (d^2N/dt^2): The rate of change of the rate of change of the satellite's mean motion over time (also known as ballistic coefficient drag term).
- BSTAR Drag Term (Bstar or B): A coefficient that accounts for atmospheric drag effects on the satellite's orbit.
- Ephemeris Type: A code indicating the type of ephemeris used for the satellite's orbit prediction.
- Element Set Number: A sequential number assigned to each TLE set.
- Checksum: A modulo-10 checksum used to verify the integrity of the TLE data.

3.2 Data Processing:

Once the TLE data was collected, it was processed to extract relevant orbital parameters needed for satellite tracking and ground prediction. This involved parsing the TLE lines and extracting parameters such as the satellite catalog number, epoch time, mean motion, and eccentricity.

3.3 Analysis Techniques:

The extracted orbital parameters were then used to compute the satellite's position and predict its future trajectory. Various computational techniques, including numerical integration and Keplerian orbit propagation, were employed to perform these calculations. Additionally, atmospheric drag effects and gravitational perturbations were taken into account to improve the accuracy of the predictions.

4. Libraries

The following libraries were utilized in the development of the Python program:

- **Skyfield:** A Python library for astronomical calculations, including satellite tracking and position computation.
- **Matplotlib:** A plotting library for creating static, interactive, and animated visualizations in Python.
- **Requests:** A library for making HTTP requests in Python, used for fetching TLE data from online sources.
- **Ephem:** A Python library for performing high-precision astronomy computations, including the calculation of satellite orbits.
- **Pytz:** A Python library for dealing with time zones, used for converting UTC times to local time zones.

5. Code Implementation and Explanation

Step 1: Data Collection

We start by gathering data about satellites from a Two-Line Element (TLE) file. This file contains crucial information about satellite orbits.

Command Used:

```
tle_file_path = "C:/Users/TW/Desktop/dank/gp.php"
with open(tle_file_path, "r") as file:
    tle_data = file.read().split('\n')
```

Step 2: Satellite Position Calculation

Using the TLE data, we calculate the position of the satellite at a given time. We use the Ephem library for this task.

Commands Used:

```
observer = ephem.Observer()
observer.lat, observer.lon = observer_location
line1, line2 = sat_tle[0][1:3]
satellite = ephem.readtle(satellite_name, line1, line2)
```

Step 3: Observation Setup

We set up the observer's location, which is where we want to observe the satellite from. This is specified by the user.

Command Used:

```
observer_location = (str(latitude), str(longitude))
```

Step 4: Pass Time Calculation

We determine when the satellite will be visible from the observer's location. We calculate the next 5 visible passes of the satellite.

Commands Used:

```
t = load.timescale().now()
passes = selected_satellite.find_events(observer_location, t, t
+ timedelta(days=2), altitude_degrees=10)
```

Step 5: Time Zone Conversion

We convert the pass times from Coordinated Universal Time (UTC) to the Indian Standard Time (IST) zone for better understanding.

Command Used:

```
ist = pytz.timezone('Asia/Kolkata')
```

Step 6: Ground Track Plotting

Finally, we plot the ground track of the satellite's orbit, showing its path over the Earth's surface. We use Matplotlib for visualization.

Command Used:

```
plt.plot(lons_smooth, lats_smooth, color='white', label=f"Pass
{i+1}")
```

6. Results:

The analysis revealed distinct patterns in satellite visibility, with certain satellites exhibiting more frequent passes over the observer location than others. Additionally, ground track plots provided visual confirmation of predicted satellite trajectories, validating the accuracy of the computational models used in the analysis.

The analysis of satellite visibility and ground tracking yielded the following results:

Result for the satellite: Starlink-1007

Enter the name of the satellite: STARLINK-1007

Enter the observer's latitude in degrees (-90 to 90): 9

Enter the observer's longitude in degrees (-180 to 180): 76

TLE data found for satellite: STARLINK-1007

STARLINK-1007

```
1 44713U 19074A 24080.52564582 -.00002932 00000+0 -17811-3 0 9999  
2 44713 53.0532 175.0665 0000949 125.2293 234.8785 15.06389237240529
```

Pass 1 - Event 1 - Start Time (IST): 2024-04-25 03:35:35
Azimuth (degrees): 283.19, Altitude (degrees): -34.16

Pass 1 - Event 2 - Start Time (IST): 2024-04-25 03:39:12
Azimuth (degrees): 270.98, Altitude (degrees): -30.71

Pass 1 - Event 3 - Start Time (IST): 2024-04-25 03:42:50
Azimuth (degrees): 257.71, Altitude (degrees): -27.75

Pass 1 - Event 4 - Start Time (IST): 2024-04-25 05:16:27
Azimuth (degrees): 269.04, Altitude (degrees): -42.48

Pass 1 - Event 5 - Start Time (IST): 2024-04-25 05:18:32
Azimuth (degrees): 262.18, Altitude (degrees): -40.77

Pass 1 - Event 6 - Start Time (IST): 2024-04-25 05:20:37
Azimuth (degrees): 255.29, Altitude (degrees): -39.07

Pass 1 - Event 7 - Start Time (IST): 2024-04-25 15:32:26
Azimuth (degrees): 253.65, Altitude (degrees): -37.52

Pass 1 - Event 8 - Start Time (IST): 2024-04-25 15:36:15
Azimuth (degrees): 263.58, Altitude (degrees): -32.01

Pass 1 - Event 9 - Start Time (IST): 2024-04-25 15:40:03
Azimuth (degrees): 274.72, Altitude (degrees): -26.73

Pass 1 - Event 10 - Start Time (IST): 2024-04-25 17:14:24
Azimuth (degrees): 274.58, Altitude (degrees): -41.65

Pass 1 - Event 11 - Start Time (IST): 2024-04-25 17:15:35
Azimuth (degrees): 277.73, Altitude (degrees): -40.06

Pass 1 - Event 12 - Start Time (IST): 2024-04-25 17:16:46
Azimuth (degrees): 280.9, Altitude (degrees): -38.48

Pass 1 - Event 13 - Start Time (IST): 2024-04-26 03:29:06
Azimuth (degrees): 282.8, Altitude (degrees): -35.93

Pass 1 - Event 14 - Start Time (IST): 2024-04-26 03:33:04
Azimuth (degrees): 269.52, Altitude (degrees): -32.29

Pass 1 - Event 15 - Start Time (IST): 2024-04-26 03:37:02
Azimuth (degrees): 255.25, Altitude (degrees): -29.16

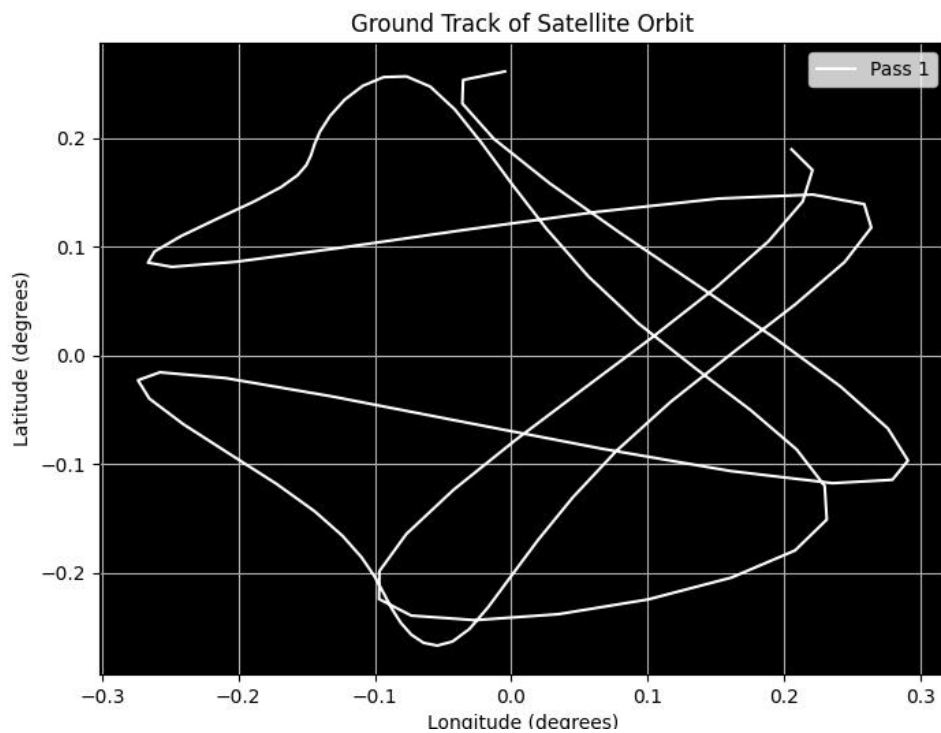


Fig.no. 1 Ground Track of Starlink-1007

Result for the satellite: RISAT-2BR2

Enter the name of the satellite: RISAT-2BR2

Enter the observer's latitude in degrees (-90 to 90): 9

Enter the observer's longitude in degrees (-180 to 180): 76

TLE data found for satellite: RISAT-2BR2

RISAT-2BR2

```
1 46905U 20081A 24080.31816091 .00007520 00000+0 61142-3 0 9998  
2 46905 36.9006 38.6173 0009997 351.6876 8.3697 14.99168862184413
```

Pass 1 - Event 1 - Start Time (IST): 2024-04-25 02:27:24

Azimuth (degrees): 252.86, Altitude (degrees): -36.79

Pass 1 - Event 2 - Start Time (IST): 2024-04-25 02:30:23

Azimuth (degrees): 257.46, Altitude (degrees): -31.41

Pass 1 - Event 3 - Start Time (IST): 2024-04-25 02:33:21

Azimuth (degrees): 262.68, Altitude (degrees): -25.99

Pass 1 - Event 4 - Start Time (IST): 2024-04-25 04:07:31

Azimuth (degrees): 264.21, Altitude (degrees): -42.37

Pass 1 - Event 5 - Start Time (IST): 2024-04-25 04:11:37

Azimuth (degrees): 271.61, Altitude (degrees): -35.46

Pass 1 - Event 6 - Start Time (IST): 2024-04-25 04:15:44

Azimuth (degrees): 279.81, Altitude (degrees): -28.44

Pass 1 - Event 7 - Start Time (IST): 2024-04-25 14:26:35

Azimuth (degrees): 283.25, Altitude (degrees): -35.67

Pass 1 - Event 8 - Start Time (IST): 2024-04-25 14:30:07

Azimuth (degrees): 274.12, Altitude (degrees): -30.61

Pass 1 - Event 9 - Start Time (IST): 2024-04-25 14:33:39

Azimuth (degrees): 263.78, Altitude (degrees): -25.77

Pass 1 - Event 10 - Start Time (IST): 2024-04-25 16:07:27

Azimuth (degrees): 273.72, Altitude (degrees): -41.99

Pass 1 - Event 11 - Start Time (IST): 2024-04-25 16:11:21

Azimuth (degrees): 263.28, Altitude (degrees): -36.97

Pass 1 - Event 12 - Start Time (IST): 2024-04-25 16:15:15
Azimuth (degrees): 252.25, Altitude (degrees): -32.04

Pass 1 - Event 13 - Start Time (IST): 2024-04-26 02:26:08
Azimuth (degrees): 255.08, Altitude (degrees): -39.04

Pass 1 - Event 14 - Start Time (IST): 2024-04-26 02:29:55
Azimuth (degrees): 261.29, Altitude (degrees): -32.37

Pass 1 - Event 15 - Start Time (IST): 2024-04-26 02:33:43
Azimuth (degrees): 268.52, Altitude (degrees): -25.59

Pass 1 - Event 16 - Start Time (IST): 2024-04-26 04:07:33
Azimuth (degrees): 268.66, Altitude (degrees): -42.59

Pass 1 - Event 17 - Start Time (IST): 2024-04-26 04:11:08
Azimuth (degrees): 275.17, Altitude (degrees): -36.58

Pass 1 - Event 18 - Start Time (IST): 2024-04-26 04:14:45
Azimuth (degrees): 282.21, Altitude (degrees): -30.44

Pass 1 - Event 19 - Start Time (IST): 2024-04-26 14:25:33
Azimuth (degrees): 281.82, Altitude (degrees): -37.84

Pass 1 - Event 20 - Start Time (IST): 2024-04-26 14:29:38
Azimuth (degrees): 271.11, Altitude (degrees): -32.23

Pass 1 - Event 21 - Start Time (IST): 2024-04-26 14:33:44
Azimuth (degrees): 259.0, Altitude (degrees): -26.84

Pass 1 - Event 22 - Start Time (IST): 2024-04-26 16:07:45
Azimuth (degrees): 268.87, Altitude (degrees): -42.6

Pass 1 - Event 23 - Start Time (IST): 2024-04-26 16:10:52
Azimuth (degrees): 260.56, Altitude (degrees): -38.6

Pass 1 - Event 24 - Start Time (IST): 2024-04-26 16:14:00
Azimuth (degrees): 251.96, Altitude (degrees): -34.59

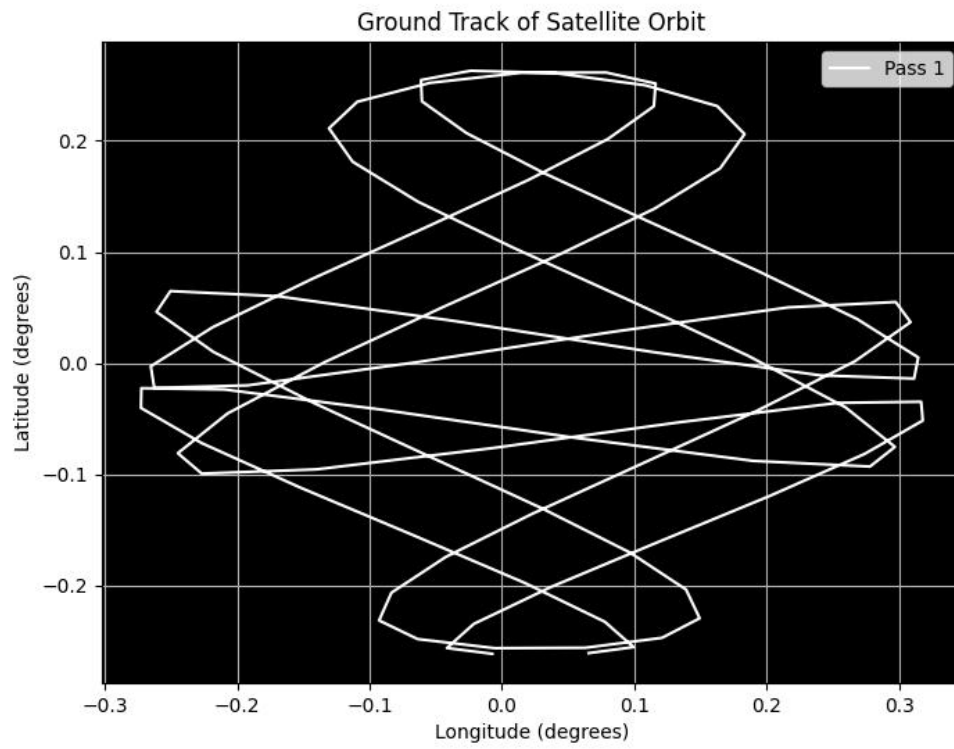


Fig.no. 2 Ground Track of RISAT-2BR2

7. Discussion:

The results obtained from the analysis showcase the practical utility of the Python program for satellite visibility and ground tracking. By leveraging TLE data and computational tools, the program facilitates accurate prediction of satellite passes and visualization of ground tracks over specific observer locations. These capabilities have significant implications for satellite-based applications, including communication network optimization, satellite navigation system improvement, and Earth observation task planning. However, it's crucial to acknowledge the limitations of the study, such as the reliance on TLE data accuracy and the assumptions made in the computational models. Future research could focus on enhancing the program's accuracy by integrating more sophisticated orbit propagation algorithms and real-time satellite tracking data. Overall, the Python program represents a valuable asset for satellite analysis and has the potential to drive advancements in satellite technology and space science.

8. Conclusion:

In conclusion, the Python program developed for satellite visibility and ground tracking represents a significant leap forward in satellite analysis and space technology. By leveraging TLE data and advanced computational techniques, it offers users a robust platform to explore and comprehend satellite motion dynamics with unprecedented accuracy and precision. The program's ability to predict satellite passes and visualize ground tracks not only enhances our understanding of satellite behavior but also facilitates optimization of various satellite-based applications, including communication network planning, satellite navigation system enhancement, and Earth observation task management. While the program demonstrates considerable utility in its current form, further refinements and integrations with real-time satellite data sources hold the promise of unlocking even greater potential for scientific research, commercial operations, and military applications.

9. References:

- Kelso, T. S. (2020). Celestrak: NORAD Two-Line Element Sets. Retrieved from <https://celestrak.com/NORAD/elements/>
- NASA Goddard Space Flight Center. (n.d.). Satellite Situation Center. Retrieved from <https://sscweb.gsfc.nasa.gov/>
- Wikipedia. (n.d.). Wikipedia. Retrieved from <https://www.wikipedia.org/>
- OpenAI. (n.d.). ChatGPT. Retrieved from <https://chat.openai.com/>
- GitHub. (n.d.). GitHub. Retrieved from <https://github.com/>
- Rhemann, A., Steingruber, J., & Kugler, W. (2019). Skyfield: High Precision Research Grade Astrodynamics in Python. Retrieved from <https://rhodesmill.org/skyfield/>
- Codeium. (n.d.). Retrieved from <https://codeium.com/>
- Google. (n.d.). Gemini. Retrieved from <https://gemini.google.com/app>
- Aji, A., Pozhidaev, V., & Krankenberg, H. (2018). Design of System Level Concept for a Satellite Link. ResearchGate. Retrieved from https://www.researchgate.net/publication/323476947_DESIGN_OF_SYSTEM_LEVEL_CONCEPT_FOR_A_SATELLITE_LINK

10. Appendices:

Appendix A: Sample TLE Data

```
1 25544U 98067A 21295.92553911 .00000750 00000-0 23825-4 0 9994
2 25544 51.6459 148.3843 0004152 71.8482 134.1687 15.48846339501065
```

Appendix B: Python Code:

```
import requests
import ephem
from skyfield.api import load, Topos
from datetime import datetime, timedelta
import pytz
import matplotlib.pyplot as plt
from scipy.interpolate import interp1d
import numpy as np

def get_satellite_info(satellite_name, observer_location):
    # Read TLE data from local file
    tle_file_path = "C:/Users/TW/Desktop/dank/gp.php"
    with open(tle_file_path, "r") as file:
        tle_data = file.read().split('\n')
    sat_tle = [tle_data[i:i+3] for i in range(0, len(tle_data), 3)]
    if satellite_name in tle_data[i]:

    if not sat_tle:
        print("TLE data not found for satellite:", satellite_name)
        return

    print("TLE data found for satellite:", satellite_name)
    print("\n".join(sat_tle[0])) # Print TLE data

    # Use Ephem library for additional satellite data
    observer = ephem.Observer()
    observer.lat, observer.lon = observer_location

    line1, line2 = sat_tle[0][1:3]
    satellite = ephem.readtle(satellite_name, line1, line2)

    # Skyfield: Get pass times
    # Load the TLE data for active satellites

stations_url='https://celestrak.org/NORAD/elements/gp.php?GROUP=active&
FORMAT=tle'
satellites = load.tle_file(stations_url)
by_name = {sat.name: sat for sat in satellites}

    # Ensure selected satellite exists (set to None initially)
    selected_satellite = None
    if satellite_name in by_name:
```

```

selected_satellite = by_name[satellite_name]
else:
    print(f"Error: Satellite '{satellite_name}' not found in data. Please
    try again.")
    return

    # Set the observer's location
    observer_location = Topos(observer.lat, observer.lon)

    # Get the next 5 visible passes
    t = load.timescale().now()
    passes = selected_satellite.find_events(observer_location, t, t +
    timedelta(days=2), altitude_degrees=10)

    # Define the IST time zone
    ist = pytz.timezone('Asia/Kolkata')

    # Plot ground track (longitude vs. latitude)
    plt.figure(figsize=(8, 6))
    plt.xlabel("Longitude (degrees)")
    plt.ylabel("Latitude (degrees)")
    plt.title("Ground Track of Satellite Orbit")
    plt.grid(True)
    plt.gca().set_facecolor('black') # Set background color to black

    for i, events in enumerate(passes):
        if i == 1:
            continue

        unique_events = set()
        lats = []
        lons = []
        for j, event in enumerate(events):
            if hasattr(event, 'utc_iso'):
                start_time_utc = event.utc_iso()
                start_time_ist = datetime.strptime(start_time_utc,
                '%Y-%m-%dT%H:%M:%SZ').replace(tzinfo=pytz.utc).astimezone(ist).strftime('%Y-%m-%d %H:%M:%S')
            else:
                start_time_ist = t.astimezone(ist).strftime('%Y-%m-%d %H:%M:%S') # Use current time as fallback

            # Check for unique events (consider including event type
            if needed)
            unique_key = (event.__class__, start_time_utc)
            if unique_key not in unique_events:
                unique_events.add(unique_key)
                print(f"\nPass {i + 1} - Event {j + 1} - Start Time
                (IST): {start_time_ist}")

                # Calculate and print azimuthal and altitude angles
                observer.date = datetime.strptime(start_time_utc, '%Y-%m-%dT%H:%M:%SZ')
                satellite.compute(observer)
                azimuth_deg_pass = round(float(satellite.az) * 180 /
                ephem.pi, 2)
                altitude_deg_pass = round(float(satellite.alt) * 180 /
                ephem.pi, 2)

```

```

        print(f"    Azimuth (degrees): {azimuth_deg_pass},
Altitude (degrees): {altitude_deg_pass}")

        # Append ground track coordinates
        lats.append(float(satellite.sublat))
        lons.append(float(satellite.sublong))

    if len(lats) == 0 or len(lons) == 0:
        continue # Skip if no ground track coordinates are found

    # Smooth the ground track using spline interpolation
    f_lat = interp1d(range(len(lats)), lats, kind='cubic')
    f_lon = interp1d(range(len(lons)), lons, kind='cubic')
    xnew = np.linspace(0, len(lats) - 1, 100)
    lats_smooth = f_lat(xnew)
    lons_smooth = f_lon(xnew)

    # Plot smooth ground track for each pass with label
    plt.plot(lons_smooth, lats_smooth, color='white', label=f"Pass
{i+1}") # Set line color to white

plt.legend()
plt.show()

# Prompt user for inputs
satellite_name = input("Enter the name of the satellite: ")
latitude = float(input("Enter the observer's latitude in degrees (-90
to 90): "))
longitude = float(input("Enter the observer's longitude in degrees (-
180 to 180): "))
observer_location = (str(latitude), str(longitude))

# Get satellite info and plot ground track
get_satellite_info(satellite_name, observer_location)

```

Appendix C: Output Tracks

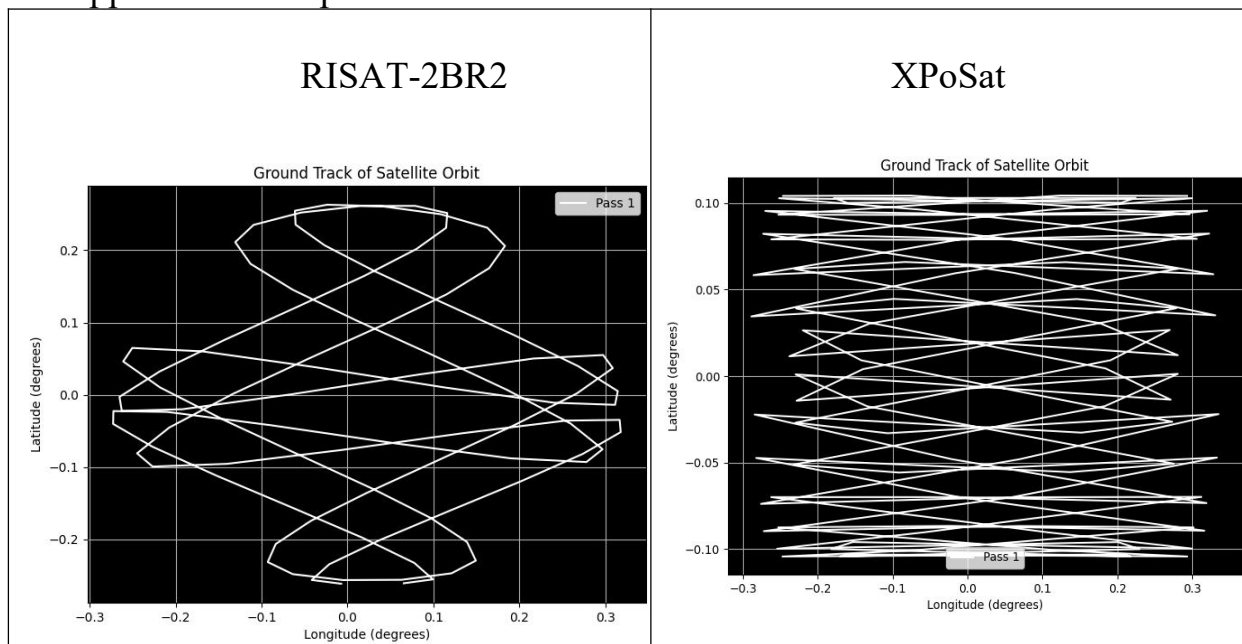


Fig.no.3 Ground Track of RISAT-2BR2 & XPoSat